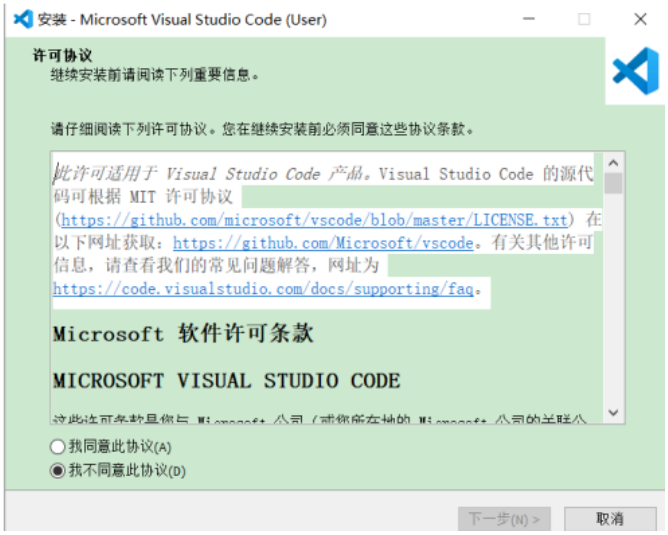
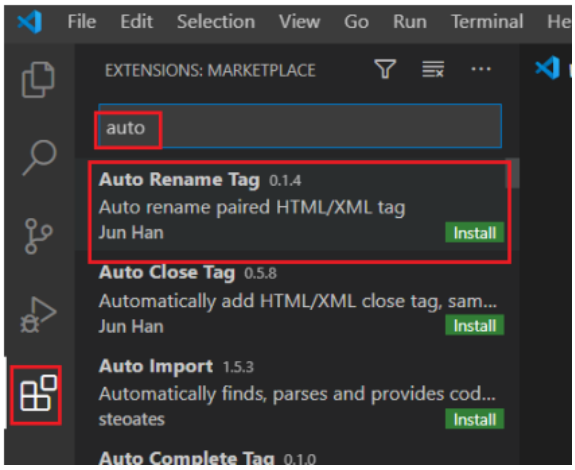
前端-ES6

- 安装vscode开发前端



- 安装插件：

```
Auto Close tag
 Auto Rename Tag
 Chinese(simplified)Language Pack for Visual Studio Code
 ESLint
 Html Css Support
 Html Snippets
 Javascript(ES6)code snippets
 Live Server
Open in browser
vetur
```



1.1 *ES6讲解*

1.1.1 简介

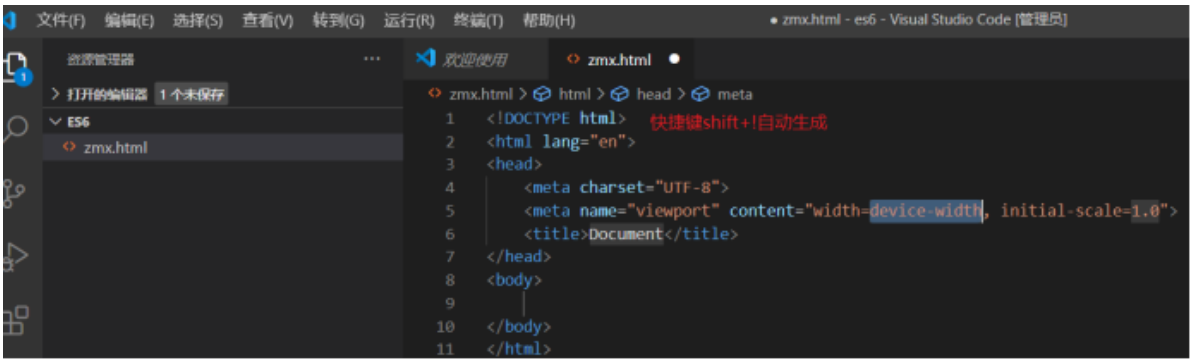
ES6(ECMAScript6.0)：是通过ECMA-262标准化的脚本程序语言，是JavaScript语言的下一代标准。2015.6月发布。

ECMAScript是浏览器脚本语言的规范，而各种js语言就是这种规范的具体实现，如：javascript.

1.1.2 ES6新特性，声明变量

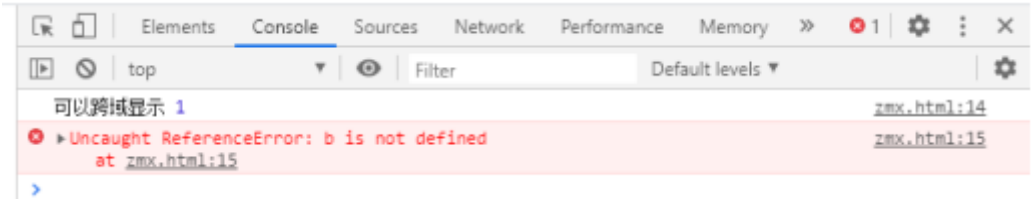
- 创建一个项目



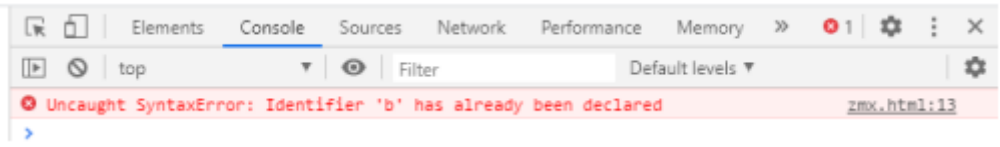


- let声明变量

```
zmx.html > html > body > script
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        {
            var a = 1;
            let b = 2;
        }
        console.log("可以跨域显示",a);
        console.log("不能跨域显示",b);
    </script>
</body>
</html>
```



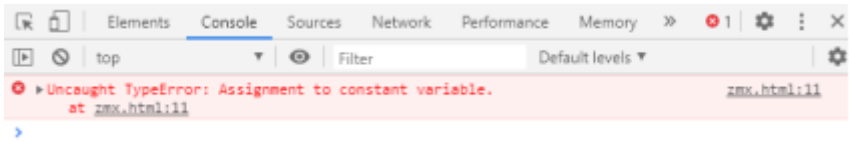

```
zmx.html > html > head
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
        //可以同一个变量名称可以声明多次
        var a = 1;
        var a = 100;
        //不可以声明多次
        let b = 2;
        let b = 20
        console.log(a);
        console.log(b);
    </script>
</body>
</html>
```



```
<script>
    /*
    var 会变量提升
    let 不存在变量提升
    */
   console.log(a);//underfined表示未赋值
   var a = 10;
   console.log(b);//is not defined表示未定义
   let b = 10;
</script>
```

- const声明变量(只读变量)

```
<script>
    const a = 10;//不允许修改
    a = 100;
</script>
```



- 结构表达式

```
<body>
    <script>
      //数组结构
      let arr = [1,2,3];
      let [a,b,c] = arr;
      console.log(a,b,c);
    </script>
</body>
```

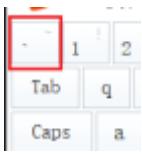
```
<script>
      //对象结构
      const person = {
          name:"",
          age:11,
          language:['hadoop','hive','hbase']
      }
      const {name,age,language} = person;
      console.log(name,age,language);
      //或者使用别名称
      const {name:a,age:b,language:la} = person;
      console.log(a,b,la);
    </script>
```

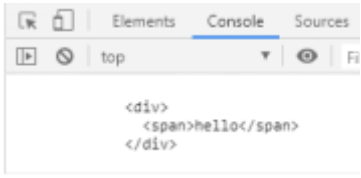
- 字符串扩展

```
<body>
    <script>
       let str = "hello.txt";
       console.log(str.startsWith("he"));//true
       console.log(str.endsWith("txt"));//true
       console.log(str.includes("e"));//是否包含：true
    </script>
</body>
```

- 字符串模板

```
<body>
    <script>
       let str = `
          <div>
            <span>hello</span>
          </div>
       `
       console.log(str);
    </script>
</body>
```





- 字符串插入变量：${变量名称}

```
<body>
    <script>
        const name = 'leo';
       let str = `
          <div>
            <span>hello:${name}</span>
          </div>
       `
       console.log(str);
    </script>
</body>
</html>
```

- 调用函数

```
<body>
    <script>
        function aa(){
            return '这是一个函数';
        }
       let str = `
          <div>
            <span>hello:${aa}</span>
          </div>
       `
       console.log(str);
    </script>
</body>
```

1.1.3函数的优化

- 函数的默认值

```
<body>
    <script>
        function aa(a,b=1){
            return a+b;
        }
       console.log(aa(10));
    </script>
</body>
```

- 不定参数

```
<body>
    <script>
        function aa(...params){
           console.log(params.length);
        }
       aa(1,2,'abc');
    </script>
</body>
```

- 匿名函数

```
<body>
    <script>
       const fun=param=>console.log('参数=',param);
       //执行
       fun(100);
    </script>
</body>

<body>
    <script>
       const fun=(x,y)=>x+y;
       //执行
       console.log(fun(10,1));;
    </script>
</body>
```

- 多语句必须添加return关键字

```
<body>
    <script>
       const fun=(x,y)=>{
           if(x==1){
               return x+y;
           }else{
               return x+y+10
           }
       };
       //执行
       console.log(fun(10,1));
    </script>
</body>
```

- 传递对象

```
<body>
    <script>
       const person={
           name:'leo',
           sex:'boy',
           language:['hadoop','hive','hbase']
       }

       const fun=(obj)=>console.log(obj.name);
       //执行
       fun(person);
    </script>
</body>
```

- 也可以使用'结构'的方式

```
<body>
    <script>
       const person={
           name:'leo',
           sex:'boy',
           language:['hadoop','hive','hbase']
       }

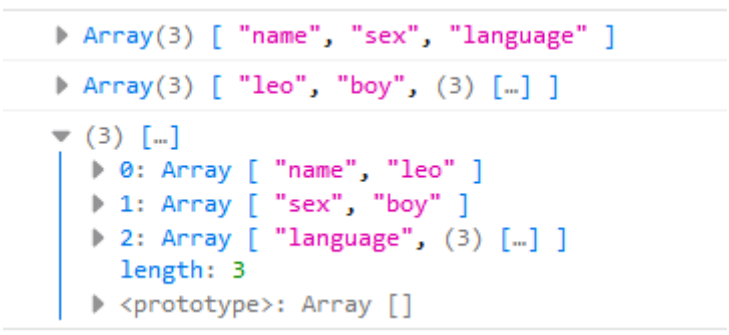
       const fun=({name})=>console.log(name);
       //执行
       fun(person);
    </script>
</body>
```

1.1.4 对象优化

- Object中的方法

```
<body>
    <script>
       const person={
           name:'leo',
           sex:'boy',
           language:['hadoop','hive','hbase']
       }

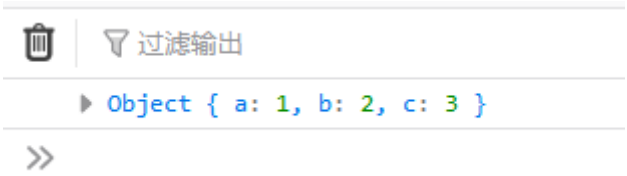
      console.log(Object.keys(person));
      console.log(Object.values(person));
      console.log(Object.entries(person));
    </script>
</body>
```



- 合并对象

```
<body>
    <script>
       const target={a:1};
       const source1={b:2};
       const source2={c:3};

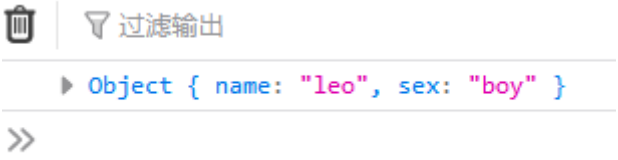
      console.log(Object.assign(target,source1,source2));
    </script>
</body>
```



- 声明对象的简写

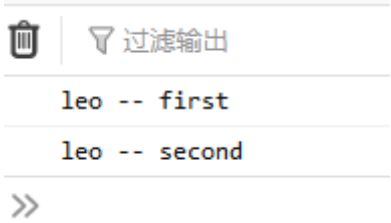
```
<body>
    <script>
       const name='leo';
       const sex='boy';
       //创建对象
       const person={name,sex};

      console.log(person);
    </script>
</body>
```



- 对象函数属性简写

```
<body>
    <script>
        let person={
            name:"leo",
            eat:(par)=>console.log(person.name,"--",par),
            eat1(par){console.log(this.name,"--",par);}
        }
        //调用
        person.eat('first');
        person.eat1('second');
    </script>
</body>
```

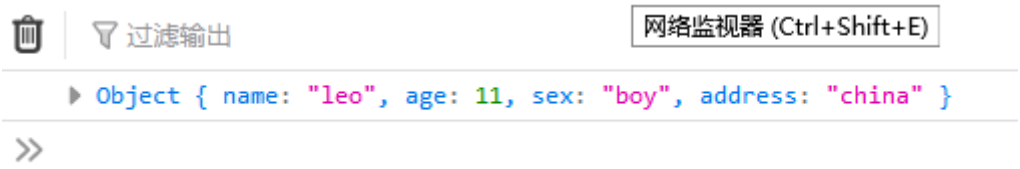


- 深拷贝

```
<body>
    <script>
        let person={name:"leo",age:11}
        //深拷贝赋值
        let emp = {...person}
        console.log(emp);
    </script>
</body>
```

- 合并对象

```
<body>
    <script>
        let person={name:"leo",age:11}
        let person1={sex:"boy",address:'china'}
        //深拷贝赋值
        let emp = {...person,...person1}
        console.log(emp);
    </script>
</body>
```



1.1.5数组的新增方法

- map方法

```
<body>
    <script>
      let arr = [1,2,3,4,5,6];
      //每一个元素都乘以2
      let a = arr.map((item)=>item*2);
      //输出
      console.log(a);
    </script>
</body>
```

- reduce方法

```
<body>
    <script>
      let arr = [1,2,3,4,5,6];
      //求和
      let result = arr.reduce((x,y)=>x+y);
      //输出
      console.log(result);
    </script>
</body>
```

1.1.6 promise优化异步操作

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Document</title>
    <script src="https://cdn.bootcss.com/jquery/3.4.1/jquery.min.
js"></script>
</head>
<body>
```

```
    <script>
        //http://jsonplaceholder.typicode.com/users/1
        var str = '{"name":"leo","sex":"boy","age":"11"}';
        //使用Promise封装异步操作
        let xiudun = new Promise((resolve,reject)=>{
            $.ajax({
                url:"https://cdn.bootcss.com/jquery/3.4.1/jquery.
min.js",
                success:function(data){
                    resolve(data);
                },
                error:function(err){
                    reject(err);
                }
            })
        });
        //获取成功返回的数据
        xiudun.then((obj)=>{console.log("获取成功后的数据：",obj)})
                                    .catch((err)=>{console.log("失败
的数据:"+err)});
    </script>
</body>
</html>
```

1.1.7 模块化

需要安装webpack才可以运行

模块功能主要有两个命令构成：export和import

- export：用于规定模块的对外接口
- import：用于导入其他模块提供的功能

- 编写函数zmx.js

```
const util = {
    sum(x,y){
        return x+y;
    }
}
let name="leo"
let sex="boy"
//批量导出
export{util,name,sex}
//export 可以导出对象，并且，一切js变量都可以导出。如：基本类型变量、函数、数组、对象。
```

- 函数zmx1.js使用

```
//导入需要的函数或者变量
import {util,name,sex} from "./zmx1.js"
//调用方法
console.log(util.sum(1,2));
```